

Aula 3

Interface Swing estruturada e organização inicial em camadas

Desenvolvimento de Sistemas | Java Desktop com Eclipse

Produto da aula: uma tela desktop para cadastrar, listar e buscar alunos em memória.

ceub.br

De onde partimos?

A Aula 2 deixou o projeto funcionando, mas ainda com interação simplificada.



- Classe Aluno continua sendo o modelo de domínio.
- CadastroAlunos continua guardando a lista e as regras.
- A interface passa a ser uma janela com componentes e eventos.

Objetivos da aula

Ao final, o aluno deve conseguir explicar e modificar uma interface Swing simples.

1
Componentes

2
Eventos

3
Camadas iniciais

4
Validações

5
Entrega explicada

Pergunta guia: o que muda quando o usuário deixa de seguir um menu e passa a clicar em botões?

JOptionPane x JFrame

A diferença não é só visual: muda a forma de organizar a interação.

JOptionPane

Interações curtas
Fluxo linear
Ótimo para começar

JFrame

Janela persistente
Eventos de botão
Mais próximo de sistema desktop

Na Aula 3, a lista em memória continua existindo. O que muda é a camada de entrada e saída.

Organização inicial em pacotes

Começamos a separar responsabilidades sem complicar demais.

model

Aluno
dados do domínio

service

CadastroAlunos
lista e regras

view

TelaCadastroAluno
campos e botões

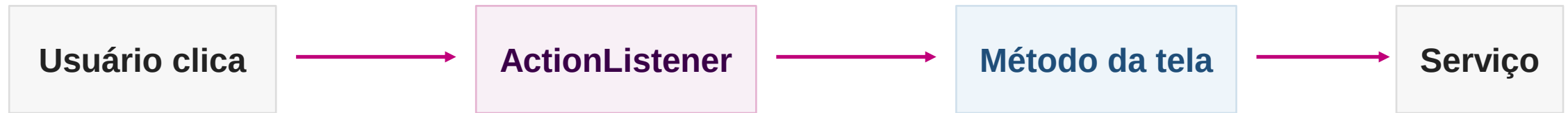
app

Principal
inicia o programa

Regra prática: botão chama método; método chama serviço; serviço manipula objetos.

Programação orientada a eventos

Em interface gráfica, o programa reage a ações do usuário.



```
botaoCadastrar.addActionListener(e -> cadastrarAluno());
```

Componentes da tela

Uma tela simples já ensina muito sobre estrutura.

Cadastro de Alunos

Nome:

Curso:

Semestre:

Cadastrar

Listar

Buscar

Área de resultado
Aluno cadastrado...
Lista de alunos...

JTEXTFIELD + JBUTTON + JTEXTAREA

Fluxo dos dados

O importante é saber onde cada responsabilidade acontece.



- A tela lê texto e mostra resposta.
- O modelo representa o aluno.
- O serviço adiciona, lista e busca.
- A lista guarda os objetos em memória.

Implementação guiada 1

Comece pelo que não depende da tela.

```
public class Aluno {  
    private String nome;  
    private int semestre;  
    private String curso;  
  
    public String gerarResumo() {  
        return nome + " - " + curso;  
    }  
}
```

```
public class CadastroAlunos {  
    private List<Aluno> alunos = new ArrayList<>();  
  
    public void adicionar(Aluno aluno) {  
        alunos.add(aluno);  
    }  
  
    public Aluno buscarPorNome(String nome) { ... }  
}
```

Implementação guiada 2

A tela cria componentes e registra eventos.

```
public class TelaCadastroAluno extends JFrame {  
    private JTextField campoNome;  
    private JTextArea areaResultado;  
    private CadastroAlunos cadastro;  
  
    public TelaCadastroAluno() {  
        cadastro = new CadastroAlunos();  
        criarComponentes();  
    }  
}
```

```
botaoCadastrar.addActionListener(e -> cadastrarAluno());  
cadastrarAluno();  
botaoListar.addActionListener(e -> listarAlunos());  
botaoBuscar.addActionListener(e -> buscarAluno());  
  
private void cadastrarAluno() {  
    // ler campos, validar, criar objeto  
}
```

Validações iniciais

Validação simples evita que o sistema aceite dados impossíveis.

- Campos obrigatórios: nome, curso e semestre.
- Semestre deve ser número.
- Semestre deve estar em uma faixa válida.
- Mensagens devem orientar o usuário, não apenas indicar erro.

```
try {  
    semestre =  
    Integer.parseInt(textoSemestre);  
} catch (NumberFormatException erro) {  
    JOptionPane.showMessageDialog(this,  
        "Digite um número válido.");  
    return;  
}
```

Nesta etapa, validar é mais importante do que construir uma interface visualmente sofisticada.

Atividade pontuada

Em dupla, implementar uma melhoria pequena e explicar o fluxo.

A
Contar alunos

B
Buscar por curso

C
Validar semestre

D
Nome duplicado

- Funcionalidade funcionando no Eclipse.
- Separação view/service preservada.
- Demonstração rápida para a professora.
- Explicação: entrada, regra, lista e saída.

Checklist de entrega

Antes de chamar a professora, verifique estes pontos.

Compila sem erro

Abre a janela

Cadastra e lista

Busca funciona

Validação mínima

Explicação clara

Se o código funciona, mas você não consegue explicar, ainda não está pronto para entrega.

Fechamento

Da próxima vez: organização de projeto e versionamento com Git

Para revisar: qual é a responsabilidade de cada pacote criado hoje?

model | service | view | app